

자료구조개론 프로젝트

KG-RAG 기반 연역 추론에서 Top-K
검색 제한이 방향 그래프의
다중 홉 경로 탐색에 미치는 영향 분석

2026년 6월 7일

성균관대학교 시스템경영공학과
2025313007 김영훈

<목차>

1. 연구 주제 및 필요성	3
1.1 연구 배경	3
1.2 연구 목적	3
1.3 프로젝트 범위	3
2. 관련 연구 및 조사 결과	4
2.1 RAG와 Top-K 검색	4
2.2 Knowledge Graph(KG)와 KG-RAG	4
2.3 연역 추론과 다중 홉 경로 탐색	4
2.4 LLM 활용 방식	4
3. 구현 내용	5
3.1 데이터셋 설계	5
3.2 자연어 규칙의 방향 edge 변환	5
3.3 Knowledge Graph 생성	6
3.4 BFS 기반 경로 탐색	6
3.5 탐색 방식 비교	6
3.6 시간/공간 복잡도 분석	7
4. 실험 및 분석 결과	8
4.1 실험 환경	8
4.2 평가 지표	8
4.3 K값에 따른 성공률 비교	9
4.4 정답 edge 보존율 비교	9
4.5 선택 edge 수와 방문 노드 수 비교	9
4.6 결과 해석	10
5. 결론 및 향후 연구 방향	10
5.1 결론	10
5.2 한계점	11
5.3 향후 개선 방향	11
6. LLM 활용 기록	11
6.1 주제 선정	11
6.2 자료 조사	11
6.3 데이터셋 생성	12
6.4 구현 및 디버깅	12
6.5 결과 해석	12

1. 연구 주제 및 필요성

1.1 연구 배경

최근 대규모 언어 모델(LLM)은 질문 응답, 문서 요약, 코드 생성, 추론 보조 등 다양한 분야에서 활용되고 있다. 그러나 LLM은 학습된 내부 지식만으로 답변을 생성하기 때문에 최신 정보나 외부 문서에 기반한 정확한 답변이 필요한 경우 한계가 있다. 이러한 한계를 보완하기 위해 사용자의 질문과 관련된 외부 문서를 검색한 뒤, 검색된 정보를 LLM의 입력에 함께 제공하는 RAG(Retrieval-Augmented Generation) 방식이 널리 사용되고 있다.

일반적인 RAG는 사용자 질문과 의미적으로 유사한 문서 조각을 Top-K 방식으로 검색한다. 즉, 전체 문서 중 질문과 가장 관련성이 높다고 판단되는 K개의 문서 또는 정보를 선택하여 LLM에 전달한다. 이 방식은 모든 문서를 입력으로 넣지 않아도 되므로 효율적이지만, 여러 단계를 거쳐 결론에 도달해야 하는 다중 홉(multi-hop) 추론 문제에서는 한계가 발생할 수 있다.

예를 들어 다음과 같은 규칙들이 있다고 하자.

비가 온다 → 땅이 젖는다
땅이 젖는다 → 운동장이 미끄럽다
운동장이 미끄럽다 → 체육 수업이 취소된다

사용자가 “비가 오면 체육 수업이 취소되는가?”라고 질문하면, 정답을 얻기 위해서는 단일 문장 하나만 검색하는 것이 아니라, 여러 규칙을 순서대로 연결해야 한다. 이때 중간 규칙인 “땅이 젖는다 → 운동장이 미끄럽다”가 검색 결과에서 누락되면 전체 지식에는 정답 경로가 존재하더라도 부분적으로 검색된 정보만으로는 결론에 도달할 수 없다.

이 문제는 자료구조 관점에서 방향 그래프의 경로 탐색 문제로 해석할 수 있다. 자연어 규칙을 $A \rightarrow B$ 형태의 방향 edge로 변환하면, 전체 지식은 Knowledge Graph로 표현될 수 있다. 이후 사용자의 질문은 시작 노드에서 목표 노드까지 도달 가능한 경로가 존재하는지 확인하는 문제로 바뀐다. 따라서 본 프로젝트에서는 LLM의 추론 능력 자체를 직접 개선하기보다는, RAG의 Top-K 검색 제한이 Knowledge Graph 기반 연역 추론 경로 탐색에 어떤 영향을 미치는지 자료구조 관점에서 분석하고자 한다.

1.2 연구 목적

본 프로젝트의 목적은 LLM 자체를 학습하거나 실제 대규모 RAG 시스템을 구현하는 것이 아니라, RAG의 Top-K 검색 제한이 KG 기반 연역 추론에 미치는 영향을 자료구조적으로 실험하는 것이다.

이를 위해 “A 이면 B” 형태의 자연어 규칙을 $A \rightarrow B$ 방향 edge로 변환하고, 이를 adjacency list 기반 Knowledge Graph로 구성하였다. 이후 사용자의 query를 start node와 target node로 해석한 뒤, BFS를 이용해 두 노드 사이의 경로 존재 여부를 탐색하였다.

실험에서는 세 가지 탐색 방식을 비교하였다. 첫째, 전체 KG를 모두 사용하는 Full KG Search를 기준으로 설정하였다. 둘째, query와 관련성이 높은 K개의 edge만 선택하는 Naive Top-K Search를 구현하였다. 셋째, Top-K로 선택된 edge 주변의 연결 edge를 추가로 포함하는 Top-K + 1-hop Expansion Search를 구현하였다.

이를 통해 전체 KG에는 정답 경로가 존재하더라도, Top-K로 구성된 부분 KG에서는 중간 edge 누락으로 인해 추론이 실패할 수 있는지 확인하고자 하였다. 또한 1-hop Expansion 방식이 이러한 경로 단절 문제를 완화할 수 있는지도 함께 분석하였다.

1.3 프로젝트 범위

본 프로젝트는 자료구조개론 과제 수준에서 구현 가능한 범위로 제한하였다. 따라서 Transformer 내부 구조, attention 계산, 딥러닝 학습, fine-tuning, CUDA 최적화 등은 다루지 않았다. 또한 실제 벡터 데이터베이스나 대규모 GraphRAG 시스템을 구현하지 않고, RAG의 Top-K 검색 과정을 edge scoring 방식으로 단순화하여 시뮬레이션하였다.

데이터셋은 직접 생성한 synthetic dataset을 사용하였다. 각 데이터는 “A 이면 B” 형태의 한국어 자연어 규칙으로 구성하였고, source와 target을 분리하여 방향 edge로 변환하였다. 논리 구조는 단순한 단방향 인과 규칙으로 제한하였으며, 부정, 대우, AND, OR, 술어 논리 등은 다루지 않았다.

구현에는 Python을 사용하였고, 주요 자료구조로는 list, dictionary, adjacency list, queue, visited set, parent map을 사용하였다. 실험에서는 Full KG Search, Naive Top-K Search, Top-K + 1-hop Expansion Search를 비교하였으며, 성공률, 정답 edge 보존율, 선택 edge 수, 방문 노드 수, 실행 시간, 시간·공간복잡도를 중심으로 분석

하였다.

2. 관련 연구 및 조사 결과

2.1 RAG와 Top-K 검색

RAG(Retrieval-Augmented Generation)는 LLM이 답변을 생성하기 전에 외부 문서나 지식을 검색하고, 검색된 정보를 함께 입력으로 제공하는 방식이다. 일반적인 LLM이 모델 내부의 파라미터 지식에 의존한다면, RAG는 외부 지식 저장소를 활용한다는 점에서 차이가 있다. 이를 통해 학습 시점 이후의 정보나 특정 문서에 기반한 답변을 생성하는 데 도움을 줄 수 있다.

RAG에서 중요한 단계는 retrieval, 즉 검색 단계이다. 전체 문서나 지식 중에서 사용자의 질문과 관련성이 높은 정보를 선택해야 하며, 이때 일반적으로 상위 K개의 문서 조각이나 지식 조각만 선택한다. 이를 Top-K 검색이라고 한다.

Top-K 검색은 입력 크기를 줄이고 처리 효율을 높일 수 있다는 장점이 있다. 그러나 다중 홉 추론에서는 질문과 직접적으로 유사한 정보뿐만 아니라, 결론으로 이어지는 중간 정보도 중요하다. 만약 정답 경로가 $A \rightarrow B \rightarrow C \rightarrow D$ 인데 Top-K 검색 결과에 $B \rightarrow C$ 가 포함되지 않는다면, 검색된 부분 지식만으로는 A에서 D까지 도달할 수 없다. 본 프로젝트에서는 이러한 현상을 Top-K 검색 제한으로 인한 경로 단절 문제로 보고 분석하였다.

2.2 Knowledge Graph와 KG-RAG

Knowledge Graph는 지식을 노드와 edge로 표현하는 그래프 구조이다. 노드는 개체, 개념, 상태 또는 명제를 나타내고, edge는 노드 사이의 관계를 나타낸다. 본 프로젝트에서는 복잡한 실제 KG 대신, “A 이면 B” 형태의 자연어 규칙을 $A \rightarrow B$ 형태의 방향 edge로 변환하여 단순화된 KG를 구성하였다.

예를 들어 “비가 온다 이면 땅이 젖는다”라는 규칙은 “비가 온다 $\rightarrow$ 땅이 젖는다”라는 방향 edge로 표현할 수 있다. 이러한 규칙들이 모이면 방향 그래프가 되고, 사용자의 질문은 시작 노드에서 목표 노드까지의 경로 존재 여부를 확인하는 문제로 바뀐다.

KG-RAG는 RAG에 Knowledge Graph의 구조적 정보를 결합하려는 접근이다. 일반적인 RAG가 주로 텍스트 조각의 유사도에 기반하여 정보를 검색한다면, KG-RAG는 지식 간의 연결 관계를 함께 활용할 수 있다. 이러한 구조는 여러 정보를 순서대로 연결해야 하는 다중 홉 추론 문제에서 특히 의미가 있다.

본 프로젝트는 실제 KG-RAG 시스템을 구현하지는 않았다. 대신 KG-RAG의 핵심 아이디어 중 하나인 “지식을 그래프 구조로 표현하고, 그래프 위에서 필요한 경로를 탐색한다”는 부분을 자료구조개론 수준으로 단순화하였다.

2.3 연역 추론과 다중 홉 경로 탐색

연역 추론은 주어진 규칙과 사실을 바탕으로 논리적인 결론을 도출하는 과정이다. 본 프로젝트에서는 복잡한 논리식을 다루지 않고, 가장 단순한 형태의 단방향 규칙만 사용하였다. 즉, “A 이면 B”라는 규칙은 $A \rightarrow B$ 라는 방향 관계로 해석하였다.

예를 들어 다음과 같은 규칙이 있다고 하자.

서버 메모리가 부족하다 $\rightarrow$ 프로세스가 느려진다

프로세스가 느려진다 $\rightarrow$ 응답 시간이 증가한다

응답 시간이 증가한다 $\rightarrow$ 사용자 요청이 실패한다

이 경우 “서버 메모리가 부족하면 사용자 요청이 실패하는가?”라는 질문은 한 번의 규칙 적용으로 해결되지 않는다. 여러 edge를 순서대로 따라가야 하므로 다중 홉 추론 문제가 된다. 자료구조 관점에서는 이는 방향 그래프에서 시작 노드부터 목표 노드까지의 경로를 찾는 문제이다.

본 프로젝트에서는 이러한 경로 탐색을 BFS로 구현하였다. BFS는 queue를 사용하여 시작 노드에서 가까운 노드부터 탐색하는 방식이다. 또한 이미 방문한 노드를 다시 방문하지 않기 위해 visited set을 사용하였다. KG에는 cycle이 존재할 수 있기 때문에 visited set은 중복 탐색과 무한 반복을 방지하는 데 필요하다. 경로를 찾은 뒤에는 parent map을 이용하여 실제 추론 경로를 복원하였다.

2.4 LLM 활용 방식

본 프로젝트에서 LLM은 실제 추론 엔진으로 사용되지 않았다. LLM에게 직접 질문을 입력하고 답변을 평가하는 방식은 모델의 언어 능력, 사전 학습 지식, 출력 변동성에 영향을 받을 수 있기 때문이다. 따라서 본 프로젝트에서는 자료구조 자체의 동작을 명확히 분석하기 위해 LLM의 답변을 평가 대상으로 삼지 않았다.

대신 LLM은 프로젝트 수행 과정의 보조 도구로 활용하였다. 주제 탐색 과정에서는 RAG, KG-RAG, GraphRAG, multi-hop reasoning과 같은 관련 개념을 조사하는 데 사용하였다. 또한 “자연어 규칙을 방향 edge로 변환하고, query를 그래프 경로 탐색 문제로 해석한다”는 아이디어를 구체화하는 데 도움을 받았다.

데이터셋 생성 과정에서도 LLM을 활용하였다. 실험용 규칙은 “A 이면 B” 형태로 구성하였고, 정답 경로 edge, distractor edge, cycle edge가 포함되도록 설계하였다. 이후 source와 target 표현이 expected_path와 정확히 일치하는지 직접 검토하여 그래프 탐색에서 노드 불일치가 발생하지 않도록 수정하였다.

구현 과정에서는 adjacency list 기반 KG 생성, BFS 경로 탐색, Top-K edge 선택, 1-hop Expansion 방식 구현, 실험 결과 저장 및 시각화 코드 작성에 LLM을 활용하였다. 따라서 본 프로젝트에서 LLM은 결과를 직접 생성하는 대상이 아니라, 주제 구체화, 데이터셋 설계, 구현 보조, 결과 해석을 돕는 도구로 사용되었다.

3. 구현 내용

3.1 데이터셋 설계

본 프로젝트에서는 실제 대규모 문서 데이터 대신, 실험 구조를 명확히 통제할 수 있는 synthetic dataset을 직접 생성하였다. 이는 Top-K 검색 제한으로 인해 다중 홉 추론 경로가 끊기는 현상을 명확하게 관찰하기 위함이다. 실제 자연어 문서에서는 정답 경로를 정확히 정의하기 어렵지만, 직접 설계한 데이터셋에서는 각 query에 대한 expected_path를 미리 지정할 수 있으므로 성공 여부와 정답 edge 보존 여부를 정량적으로 평가할 수 있다.

데이터셋은 rules_medium.csv와 queries_medium.csv로 구성하였다. rules_medium.csv에는 자연어 규칙 문장과 source, target, domain, type 정보를 저장하였다. 각 규칙은 “A 이면 B” 형태로 작성하였으며, source와 target을 이용해 방향 edge를 생성하였다. queries_medium.csv에는 query, start, target, expected_path를 저장하였다. expected_path는 해당 query에서 의도한 정답 추론 경로를 의미한다.

규칙의 type은 answer, distractor, cycle로 구분하였다. answer는 정답 경로를 구성하는 edge이고, distractor는 query와 표면적으로 관련 있어 보이지만 실제 정답 경로에는 포함되지 않는 방해 edge이다. cycle은 그래프 안에 순환 구조를 만들기 위한 edge로, BFS 탐색에서 visited set의 필요성을 확인하기 위해 포함하였다.

초기에는 small dataset을 별도로 구성하여 데이터 입력과 그래프 탐색 로직을 검증하였고, 이후 실험 결과의 비교 가능성을 높이기 위해 domain과 edge 수를 확장한 medium dataset을 최종 분석용 데이터셋으로 사용하였다.

실험 데이터셋은 총 10개의 domain으로 구성하였다. 각 domain은 9개의 rule을 포함하며, 전체 rule 수는 90개이다. 각 rule은 하나의 방향 edge로 변환되므로 전체 Knowledge Graph는 90개의 edge로 구성된다. query는 domain별 1개씩 총 10개이며, 각 query의 expected_path는 3개의 edge로 이루어진 다중 홉 경로로 설계하였다. 전체 rule 중 정답 경로를 구성하는 answer edge는 30개, distractor edge는 40개, cycle edge는 20개이다.

파일	주요 열	역할
rules_medium.csv	rule, source, target, domain, type	자연어 규칙과 방향 edge 정보 저장
queries_medium.csv	query, start, target, expected_path	질의와 정답 경로 정보 저장

이 구조를 통해 전체 Knowledge Graph에는 정답 경로가 존재하도록 하되, Top-K로 선택된 부분 KG에서는 중간 edge가 누락될 수 있는 실험 조건을 만들었다.

3.2 자연어 규칙의 방향 edge 변환

본 프로젝트에서는 자유로운 자연어 문장을 완전한 논리식으로 변환하는 복잡한 자연어처리 문제는 다루지 않았다. 대신 “A 이면 B” 형태의 패턴화된 자연어 규칙을 사용하고, 각 규칙에서 source와 target을 명시적으로 분리하였다. 예를 들어 다음 규칙은 하나의 방향 edge로 변환된다.

<비가 온다 이면 땅이 젖는다>
 → source: 비가 온다
 → target: 땅이 젖는다
 → edge: 비가 온다 → 땅이 젖는다

프로그램에서는 CSV 파일의 source와 target 컬럼을 읽어 (source, target, type) 형태의 tuple로 저장하였다. 이 방식은 실제 자연어 전체를 처리하는 것보다 단순하지만, 자연어 규칙을 그래프 자료구조로 변환하는 핵심 과정을 보

여줄 수 있다.

3.3 Knowledge Graph 생성

변환된 edge들은 adjacency list 형태의 Knowledge Graph로 저장하였다. adjacency list는 각 노드를 key로 두고, 해당 노드에서 이동 가능한 다음 노드들의 리스트를 value로 저장하는 방식이다.

예를 들어 다음과 같은 규칙들이 있다고 하자.

비가 온다 → 땅이 젖는다
땅이 젖는다 → 운동장이 미끄럽다
운동장이 미끄럽다 → 체육 수업이 취소된다

이를 adjacency list로 표현하면 다음과 같다.

```
{  "비가 온다": ["땅이 젖는다"],  
  "땅이 젖는다": ["운동장이 미끄럽다"],  
  "운동장이 미끄럽다": ["체육 수업이 취소된다"]  
}
```

adjacency list는 특정 노드에서 이동 가능한 다음 노드를 빠르게 조회할 수 있으므로 BFS와 같은 그래프 탐색 알고리즘을 구현하기에 적합하다. 본 프로젝트에서는 전체 rules 데이터를 모두 사용하여 Full KG를 생성하였고, Top-K 방식에서는 선택된 edge만 사용하여 부분 KG를 생성하였다.

3.4 BFS 기반 경로 탐색

연역 추론 과정은 방향 그래프에서 start node부터 target node까지 도달 가능한 경로가 존재하는지 확인하는 문제로 모델링하였다. 이를 위해 BFS(Breadth-First Search)를 사용하였다.

BFS는 queue를 사용하여 시작 노드와 가까운 노드부터 순서대로 탐색하는 알고리즘이다. 본 프로젝트에서는 Python의 deque를 사용하여 queue를 구현하였다. 또한 이미 방문한 노드를 다시 방문하지 않기 위해 visited set을 사용하였다. Knowledge Graph에는 cycle이 존재할 수 있으므로, visited set이 없으면 같은 노드를 반복적으로 방문하여 탐색이 비효율적이거나 무한 반복에 빠질 수 있다.

경로를 찾을 경우에는 parent map을 이용해 실제 추론 경로를 복원하였다. parent map은 각 노드가 어떤 이전 노드로부터 방문되었는지를 저장하는 dictionary이다. 목표 노드에 도달한 뒤 parent map을 역추적하면 start node부터 target node까지의 경로를 얻을 수 있다.

예를 들어 비가 온다에서 체육 수업이 취소된다까지 탐색한 결과는 다음과 같이 복원될 수 있다.

비가 온다 → 땅이 젖는다 → 운동장이 미끄럽다 → 체육 수업이 취소된다

이 경로는 단순한 성공 여부뿐만 아니라, 어떤 규칙들을 거쳐 결론에 도달했는지를 보여주기 때문에 설명 가능성 측면에서도 의미가 있다.

초기 실험 코드에서는 BFS와 DFS를 모두 구현하여 경로 탐색 가능 여부를 확인하였다. 그러나 본 프로젝트의 최종 비교 실험에서는 unweighted graph에서 가장 짧은 hop 수의 경로를 우선 탐색할 수 있는 BFS를 기준 알고리즘으로 사용하였다.

3.5 탐색 방식 비교

본 프로젝트에서는 Top-K 검색 제한이 경로 탐색에 미치는 영향을 분석하기 위해 세 가지 방식을 비교하였다.

방식	사용 edge	목적
Full KG Search	전체 edge	전체 KG에 정답 경로가 존재하는지 확인하는 기준선
Naive Top-K Search	관련도 점수가 높은 K개 edge	RAG의 Top-K 검색 제한을 단순화하여 시뮬레이션
Top-K + 1-hop Expansion Search	Top-K edge와 주변 연결 edge	Top-K에서 누락된 중간 edge를 보완

Full KG Search는 전체 rules 데이터를 모두 사용하여 KG를 만든 뒤 BFS를 수행하는 방식이다. 이 방식은 Top-K 검색 제한을 적용하지 않으므로, 전체 지식 안에 정답 경로가 실제로 존재하는지를 확인하는 기준선 역할을 한다.

Naive Top-K Search는 전체 edge 중 query와 관련성이 높다고 판단되는 상위 K개의 edge만 선택한 뒤, 선택된

edge들로 부분 KG를 만들고 BFS를 수행하는 방식이다. 본 프로젝트에서는 실제 embedding 기반 vector search를 사용하지 않고, query의 start, target, query text와 edge의 source, target 사이의 단순 관련도 점수를 이용하였다. 이 방식은 실제 RAG 검색을 완전히 재현한 것은 아니지만, Top-K 검색이 일부 edge만 선택하여 부분 그래프를 만든다는 점을 단순화한 것이다.

Top-K + 1-hop Expansion Search는 Naive Top-K 방식으로 선택된 edge에 등장한 노드들을 기준으로 주변 edge를 추가로 포함하는 방식이다. 즉, 먼저 Top-K edge를 선택한 뒤, 선택된 edge의 source와 target 노드를 수집하고, 전체 edge 중 이 노드들과 연결된 edge를 추가한다. 이후 확장된 edge 집합으로 부분 KG를 구성하고 BFS를 수행한다.

이 방식은 단순히 K를 크게 늘리는 것이 아니라, Top-K로 선택된 edge의 그래프 연결성을 활용하여 누락된 중간 edge를 복원하려는 접근이다. 다만 주변 edge를 추가로 포함하므로 Naive Top-K보다 선택 edge 수와 탐색 공간이 증가할 수 있다.

본 프로젝트의 Naive Top-K 검색은 실제 embedding 기반 검색을 사용하지 않고, query와 edge 사이의 문자열 기반 단순 점수로 시뮬레이션하였다. 점수는 edge의 source가 query start와 일치하는 경우, edge의 target이 query target과 일치하는 경우, query 문장과 edge의 source-target 사이에 공통 토큰이 있는 경우 높아지도록 설계하였다. 또한 distractor edge는 표면적으로 관련 있어 보이는 정보로 가정하여 소량의 추가 점수를 부여하였다. 이 방식은 실제 RAG 검색 성능을 재현하기 위한 것이 아니라, Top-K 제한으로 인해 일부 edge만 선택되는 상황을 자료구조적으로 관찰하기 위한 단순화된 검색 모델이다.

```
def select_top_k_edges(rules, query_start, query_target, k):
    scored_edges = []

    for edge in rules:
        score = edge_score(edge, query_start, query_target)
        scored_edges.append((score, edge))

    scored_edges.sort(key=lambda x: x[0], reverse=True)

    top_k = [edge for score, edge in scored_edges[:k]]
    return top_k
```

[코드 2] Naive Top-K edge 선택 코드

```
for source, target, edge_type in rules:
    if source in selected_nodes or target in selected_nodes:
        if (source, target) not in existing_edges:
            expanded_rules.append((source, target, edge_type))
            existing_edges.add((source, target))
```

[코드 3] Top-K로 선택된 노드 주변 edge를 확장하는 코드

3.6 시간·공간복잡도 분석

본 프로젝트에서 사용하는 주요 변수는 다음과 같다.

기호	의미
V	전체 노드 수
E	전체 edge 수
K	Top-K에서 선택하는 edge 수
V_K	Top-K 부분 KG의 노드 수
E_K	Top-K 부분 KG의 edge 수
V_X	Expansion 이후 부분 KG의 노드 수
E_X	Expansion 이후 부분 KG의 edge 수

Full KG Search는 전체 edge를 사용하여 adjacency list를 구성하므로 그래프 생성에 $O(E)$ 의 시간이 필요하다. BFS 탐색은 최악의 경우 모든 노드와 edge를 방문하므로 $O(V + E)$ 의 시간이 걸린다. 공간복잡도는 adjacency list, visited set, parent map 등을 고려하여 $O(V + E)$ 이다.

Naive Top-K Search는 모든 edge에 대해 관련도 점수를 계산한 뒤 정렬하여 상위 K개를 선택하였다. 현재 구현은 전체 edge를 정렬하는 방식이므로 Top-K 선택 비용은 $O(E \log E)$ 이다. 이후 선택된 K개의 edge로 부분 그래프를 만들고 BFS를 수행하므로 탐색 비용은 $O(V_K + K)$ 이다. 공간복잡도는 부분 그래프와 탐색 자료구조를 포함하여 $O(V_K + K)$ 이다.

Top-K + 1-hop Expansion Search는 Naive Top-K 선택 이후, 선택된 edge에 등장한 노드와 연결된 주변 edge를 추가로 탐색한다. 이를 위해 전체 edge를 한 번 더 순회하므로 expansion 과정에 $O(E)$ 의 시간이 추가된다. 확장된 edge 수를 E_X 라고 할 때, BFS 탐색 비용은 $O(V_X + E_X)$ 이다. 공간복잡도는 확장된 부분 그래프와 탐색

자료구조를 포함하여 $O(V_X + E_X)$ 이다.
세 방식의 복잡도는 다음과 같이 정리할 수 있다.

방식	시간복잡도	공간복잡도	특징
Full KG Search	$O(V + E)$	$O(V + E)$	전체 그래프 사용
Naive Top-K	$O(E \log E + V_K + K)$	$O(V_K + K)$	중간 edge 누락 가능
Top-K + Expansion	$O(E \log E + E + V_X + E_X)$	$O(V_X + E_X)$	주변 edge 확장

이론적으로 Naive Top-K와 Expansion 방식은 모두 edge 점수 계산과 정렬 과정에서 $O(E \log E)$ 의 비용이 발생한다. 그러나 실험에서 중요한 차이는 동일한 K 조건에서 정답 경로를 구성하는 edge가 얼마나 보존되는가이다. Naive Top-K 방식은 K가 작을 때 중간 edge를 누락하여 실패하는 경우가 많았고, Expansion 방식은 선택된 node 주변 edge를 추가함으로써 더 작은 K에서도 정답 경로를 복원할 수 있었다.

4. 실험 및 분석 결과

4.1 실험 환경

본 프로젝트의 실험은 Python 기반으로 구현하였다. 개발 환경은 VSCode를 사용하였으며, 주요 실험 코드는 main_expansion.py에서 실행하였다. 데이터셋은 직접 생성한 rules_medium.csv와 queries_medium.csv를 사용하였다.

rules_medium.csv에는 자연어 규칙 문장과 source, target, domain, type 정보가 저장되어 있으며, queries_medium.csv에는 각 query의 start, target, expected_path가 저장되어 있다. 실험 데이터셋은 총 10개의 domain으로 구성하였고, 각 domain에는 정답 경로를 구성하는 answer edge, 정답 경로에는 포함되지 않는 distractor edge, cycle 구조를 만들기 위한 cycle edge를 포함하였다.

실험에서는 다음 세 가지 방식을 비교하였다.

방식	설명
Full KG Search	전체 Knowledge Graph를 사용하여 BFS 경로 탐색 수행
Naive Top-K Search	관련도 점수가 높은 K개의 edge만 선택한 뒤 BFS 수행
Top-K + 1-hop Expansion Search	Top-K edge 주변의 연결 edge를 추가한 뒤 BFS 수행

Full KG Search는 전체 지식 안에 정답 경로가 존재하는지 확인하기 위한 기준선으로 사용하였다. Naive Top-K Search는 RAG의 Top-K 검색 제한을 단순화한 방식이며, Top-K + 1-hop Expansion Search는 중간 edge 누락 문제를 완화하기 위한 개선 방식이다.

초기 기능 검증 단계에서는 더 작은 규모의 small dataset을 사용하여 CSV 입력, KG 생성, BFS 경로 탐색, Top-K 선택 로직이 정상적으로 동작하는지 확인하였다. 이후 최종 비교 실험은 더 많은 domain과 edge를 포함한 medium dataset을 기준으로 수행하였다.

실험 결과는 experiment_medium_compare.csv에 상세 저장하였고, K값별 요약 결과는 summary_medium_compare.csv에 저장하였다. 이후 visualize_results.py를 이용하여 성공률, 정답 edge 보존율, 선택 edge 수, 방문 노드 수를 그래프로 시각화하였다.

4.2 평가 지표

본 프로젝트에서는 단순히 경로 탐색이 성공했는지만 보지 않고, 성공 또는 실패의 원인을 함께 분석하기 위해 여러 평가 지표를 사용하였다.

평가 지표	의미
성공률	start node에서 target node까지 경로를 찾은 query의 비율
정답 경로 일치율	탐색 결과가 expected_path와 정확히 일치한 비율
정답 edge 보존율	expected_path를 구성하는 edge 중 부분 KG에 포함된 edge의

	비율
평균 누락 edge 수	정답 경로 중 부분 KG에 포함되지 않은 edge의 평균 개수
평균 선택 edge 수	각 방식이 실제 탐색에 사용한 edge 수
평균 방문 노드 수	BFS 탐색 중 실제로 방문한 노드 수

이 중 핵심 지표는 성공률과 정답 edge 보존율이다. 성공률은 최종적으로 추론에 성공했는지를 보여주고, 정답 edge 보존율은 실패 원인이 정답 경로의 edge 누락 때문인지 확인하는 데 사용된다. 선택 edge 수와 방문 노드 수는 각 방식의 탐색 비용을 비교하기 위한 보조 지표로 사용하였다

4.3 K값에 따른 성공률 비교

첫 번째 실험에서는 K값 변화에 따라 세 방식의 경로 탐색 성공률이 어떻게 달라지는지 비교하였다. Full KG Search는 전체 Knowledge Graph를 모두 사용하므로 모든 query에서 정답 경로를 찾을 수 있었다. 따라서 성공률이 항상 1.00인 기준으로 나타났다.

반면 Naive Top-K Search는 K가 작은 구간에서 낮은 성공률을 보였다. 이는 query와 직접적으로 관련 있어 보이는 edge는 선택되었지만, 다중 홉 추론에 필요한 중간 edge가 Top-K 결과에서 누락되었기 때문이다. 실험 결과 Naive Top-K 방식은 K가 상당히 커진 뒤에야 성공률이 상승하였고, K=88 부근에서 성공률 1.00에 도달하였다. 이는 전체 edge 수에 가까운 많은 edge를 선택해야 정답 경로가 완전히 보존되었음을 의미한다.

반면 Top-K + 1-hop Expansion Search는 훨씬 작은 K에서도 높은 성공률을 보였다. 실험 결과 K=6 수준에서 성공률 1.00에 도달하였다. 이는 처음 선택된 Top-K edge 주변의 연결 edge를 확장함으로써, Naive Top-K에서 누락되었던 중간 edge가 부분 KG에 포함되었기 때문이다.

이 결과는 단순히 질문과 유사한 edge만 선택하는 방식이 다중 홉 연역 추론에는 충분하지 않을 수 있음을 보여준다. 특히 전체 지식에는 정답 경로가 존재하더라도, Top-K 부분 KG에 중간 edge가 포함되지 않으면 추론은 실패한다. 반면 그래프 연결성을 활용하여 주변 edge를 확장하면 더 작은 K에서도 정답 경로를 복원할 수 있었다.

4.4 정답 edge 보존율 비교

두 번째 실험에서는 K값에 따른 정답 edge 보존율을 비교하였다. 정답 edge 보존율은 expected_path를 구성하는 edge 중 실제 선택된 부분 KG에 포함된 edge의 비율이다. 이 지표는 경로 탐색 실패의 원인을 분석하는 데 중요하다.

Naive Top-K 방식에서는 K가 작은 구간에서 정답 edge 보존율이 낮게 나타났다. 특히 query의 start 또는 target과 직접 관련된 edge는 상대적으로 높은 점수를 받아 선택되었지만, 정답 경로의 중간 edge는 query와 직접적인 단어 겹침이 적어 늦게 선택되는 경향이 있었다. 예를 들어 $A \rightarrow B \rightarrow C \rightarrow D$ 경로에서 $A \rightarrow B$ 와 $C \rightarrow D$ 는 선택되었지만, 중간 edge인 $B \rightarrow C$ 가 누락되면 전체 경로는 단절된다.

실험 과정에서도 이러한 현상을 확인할 수 있었다. K가 충분히 커지기 전까지는 정답 edge 3개 중 1개 이상이 누락되는 경우가 많았고, 그 결과 부분 KG에서 start node부터 target node까지 도달하지 못하였다. Naive Top-K 방식이 성공률 1.00에 도달하기 위해 K=88 부근까지 필요했던 이유도 정답 경로의 마지막 중간 edge들이 늦게 포함되었기 때문이다.

Top-K + 1-hop Expansion 방식은 정답 edge 보존율이 훨씬 빠르게 상승하였다. 이는 Top-K로 선택된 edge에 등장한 노드와 연결된 주변 edge를 추가로 포함했기 때문이다. 따라서 query와 직접 유사하지 않아 Naive Top-K에서는 누락되었던 중간 edge가, Expansion 과정에서 복원될 가능성이 커졌다.

이 결과는 다중 홉 추론에서 중요한 것은 단순히 관련 있어 보이는 정보를 많이 가져오는 것이 아니라, 정답 경로를 구성하는 연결 edge가 부분 KG 안에 보존되는지임을 보여준다.

4.5 선택 edge 수와 방문 노드 수 비교

세 번째 실험에서는 각 방식이 실제로 선택한 edge 수와 BFS 탐색 중 방문한 노드 수를 비교하였다. 이 지표는 각 방식의 효율성을 분석하기 위해 사용하였다.

Full KG Search는 전체 edge를 모두 사용하므로 선택 edge 수가 가장 많다. 이 방식은 정답 경로를 안정적으로 찾을 수 있지만, 그래프 크기가 커질수록 탐색해야 할 범위가 커질 수 있다. 따라서 Full KG Search는 성공률 측면에서는 기준선 역할을 하지만, 실제 대규모 시스템에서는 모든 edge를 매번 사용하는 것이 비효율적일 수 있다.

Naive Top-K Search는 선택 edge 수가 K로 제한되므로 탐색 공간은 작다. 그러나 K가 작을 때는 정답 경로의 중간 edge가 누락되어 성공률이 낮았다. 성공률을 1.00까지 높이기 위해서는 K를 매우 크게 설정해야 했고, 실험에서는 K=88 부근에서야 모든 query에 대해 성공하였다. 이는 결국 Naive Top-K 방식이 안정적인 다중 홉 추론을 위해서는 거의 전체 KG에 가까운 edge를 선택해야 할 수 있음을 의미한다.

Top-K + 1-hop Expansion Search는 Naive Top-K보다 더 많은 edge를 선택한다. 이는 선택된 edge 주변의 연결 edge를 추가로 포함하기 때문이다. 따라서 같은 K값에서 평균 선택 edge 수는 Naive Top-K보다 크다. 그러나 실험 결과 매우 작은 K에서도 높은 성공률을 보였으므로, 단순히 K를 크게 늘리는 방식보다 그래프 연결성을 활용하는 방식이 더 효율적인 경로 복원 전략이 될 수 있음을 확인하였다.

방문 노드 수 역시 방식별 차이를 보였다. Expansion 방식은 선택 edge 수가 증가하므로 방문 가능한 노드 수도 증가할 수 있다. 그러나 Full KG Search처럼 전체 그래프를 모두 사용하는 것은 아니므로, 전체 KG보다 훨씬 작은 부분 그래프 안에서 정답 경로를 찾을 수 있었다. 따라서 Expansion 방식은 Naive Top-K와 Full KG Search의 중간적인 성격을 가진다. 즉, Naive Top-K보다 탐색 공간은 커지지만, Full KG보다 작은 그래프에서 더 높은 성공률을 얻을 수 있다.

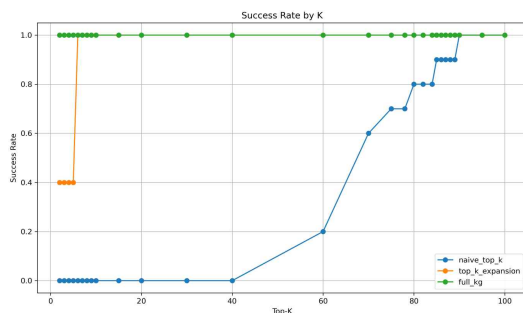
4.6 결과 해석

실험 결과를 종합하면, 전체 Knowledge Graph에는 모든 query에 대한 정답 경로가 존재하였다. Full KG Search가 모든 query에서 성공한 것은 데이터셋 설계상 정답 경로가 올바르게 포함되어 있음을 확인시켜준다. 따라서 이후 Naive Top-K와 Expansion 방식에서 발생한 실패는 정답이 전체 데이터에 없기 때문이 아니라, Top-K 검색 과정에서 필요한 edge가 부분 KG에 포함되지 않았기 때문에 발생한 것이다.

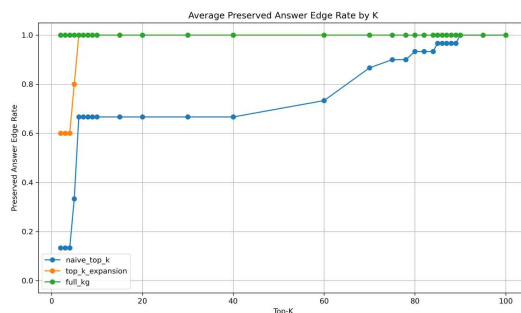
Naive Top-K Search는 RAG의 Top-K 검색 제한을 단순화한 방식이다. 이 방식은 query와 직접 관련 있어 보이는 edge를 선택하는 데는 효과적이지만, 다중 홉 추론에 필요한 중간 edge를 충분히 보존하지 못하였다. 그 결과 전체 KG에는 정답 경로가 있음에도 불구하고 부분 KG에서는 경로가 단절되는 현상이 발생하였다. 실험에서는 Naive Top-K 방식이 성공률 1.00에 도달하기 위해 K=88 부근까지 필요하였다. 이는 거의 전체 edge를 가져와야만 정답 경로가 완전히 보존되었음을 의미한다.

Top-K + 1-hop Expansion Search는 이러한 문제를 완화하였다. 이 방식은 Top-K로 선택된 edge에 등장한 노드 주변의 edge를 추가로 포함함으로써, query와 직접적으로 유사하지 않은 중간 edge를 복원할 수 있었다. 실험 결과 Expansion 방식은 K=6 수준에서 성공률 1.00에 도달하였다. 이는 그래프의 연결 구조를 활용하면 훨씬 작은 K에서도 다중 홉 추론 경로를 보존할 수 있음을 보여준다.

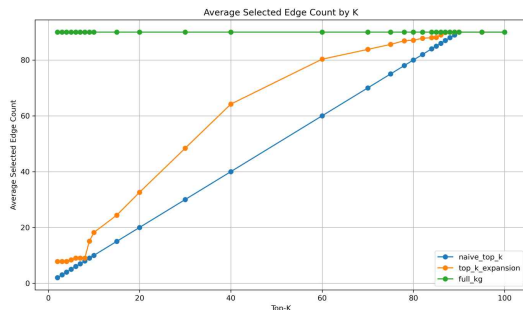
다만 Expansion 방식이 항상 더 좋은 것은 아니다. 주변 edge를 추가로 포함하기 때문에 Naive Top-K보다 선택 edge 수가 증가하고, 이에 따라 탐색 공간도 커질 수 있다. 따라서 Expansion 방식은 성공률을 높이는 대신 일정한 비용 증가를 감수하는 방식이라고 볼 수 있다. 하지만 본 실험에서는 Full KG 전체를 사용하는 것보다는 훨씬 적은 edge를 사용하면서도 높은 성공률을 얻을 수 있었기 때문에, 단순 Top-K보다 효율적인 대안이 될 수 있음을 확인하였다.



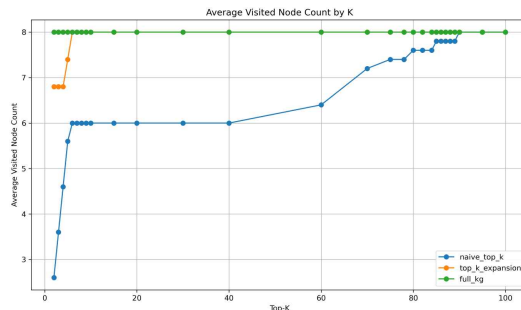
[그림 1] K값에 따른 경로 탐색 성공률 비교



[그림 2] K값에 따른 정답 edge 보존률 비교



[그림 3] K값에 따른 평균 선택 edge 수 비교



[그림 4] K값에 따른 평균 방문 노드 수 비교

5. 결론 및 향후 연구 방향

5.1 결론

본 프로젝트에서는 RAG의 Top-K 검색 제한이 Knowledge Graph 기반 다중 홉 연역 추론에 어떤 영향을 미치는지 자료구조 관점에서 분석하였다. 자연어 규칙을 $A \rightarrow B$ 형태의 방향 edge로 변환하고, 이를 adjacency list 기반 Knowledge Graph로 구성한 뒤 BFS를 통해 start node에서 target node까지의 경로를 탐색하였다.

실험 결과, Full KG Search에서는 전체 Knowledge Graph를 사용하기 때문에 모든 query에서 정답 경로를 찾을 수 있었다. 반면 Naive Top-K Search에서는 query와 직접 관련된 edge는 선택되었지만, 다중 홉 추론에 필요한 중간 edge가 누락되어 경로 탐색에 실패하는 경우가 많았다. 특히 Naive Top-K 방식은 성공률 1.00에 도달하기 위해 거의 전체 edge에 가까운 K값이 필요하였다.

이를 보완하기 위해 Top-K + 1-hop Expansion Search를 구현하였다. 이 방식은 Top-K로 선택된 edge에 등장한 노드 주변의 edge를 추가로 포함하여, 누락된 중간 edge를 복원할 가능성을 높였다. 실험 결과 Expansion 방식은 Naive Top-K보다 훨씬 작은 K에서도 높은 성공률을 보였다.

따라서 본 프로젝트를 통해 RAG 기반 추론에서 중요한 것은 전체 데이터 안에 정답이 존재하는지 여부만이 아니라, 검색된 부분 지식 안에 정답 경로를 구성하는 edge가 실제로 보존되는지임을 확인할 수 있었다. 또한 단순한 Top-K 검색보다 그래프 연결성을 고려한 확장 방식이 다중 홉 추론에 더 적합할 수 있음을 확인하였다.

5.2 한계점

본 프로젝트는 실제 대규모 RAG 시스템이나 실제 LLM 추론 과정을 구현한 것은 아니다. RAG의 검색 과정은 embedding 기반 vector search가 아니라, edge scoring을 이용해 단순화하여 시뮬레이션하였다. 따라서 실제 RAG 시스템의 검색 성능이나 LLM의 답변 품질을 직접 평가했다고 보기는 어렵다.

또한 데이터셋은 직접 생성한 synthetic dataset을 사용하였다. 이 방식은 정답 경로를 명확히 알 수 있다는 장점이 있지만, 실제 자연어 문서처럼 표현이 다양하거나 불완전한 상황은 충분히 반영하지 못한다. 자연어 규칙 역시 “A 이면 B” 형태로 제한하였기 때문에, 복잡한 조건문이나 부정, 대우, AND, OR와 같은 논리는 다루지 않았다.

마지막으로 Top-K + 1-hop Expansion 방식은 성공률을 높였지만, 주변 edge를 추가로 포함하기 때문에 선택 edge 수와 탐색 공간이 증가할 수 있다. 따라서 확장 범위가 커질 경우 효율성 문제가 발생할 수 있다.

5.3 향후 개선 방향

향후에는 실제 embedding 기반 검색 방식을 적용하여 현재의 단순 edge scoring 방식과 비교해볼 수 있다. 예를 들어 TF-IDF, sentence embedding, vector similarity 등을 사용하면 실제 RAG 검색 과정에 더 가까운 실험을 수행할 수 있다.

또한 데이터셋을 더 다양하게 확장할 수 있다. 현재는 “A 이면 B” 형태의 단순 규칙만 사용하였지만, 이후에는 여러 조건이 결합된 규칙이나 더 긴 추론 경로를 포함한 데이터셋으로 실험을 확장할 수 있다.

마지막으로 Expansion 방식도 개선할 수 있다. 현재는 선택된 edge 주변을 단순히 1-hop 확장하였지만, 향후에는 목표 노드와의 거리, edge 중요도, 경로 가능성 등을 고려하여 더 효율적으로 확장하는 방식을 설계할 수 있다. 이를 통해 탐색 공간을 과도하게 늘리지 않으면서도 다중 홉 추론 경로를 더 잘 보존하는 방법을 연구할 수 있다.

6. LLM 활용 기록

6.1 주제 선정

프로젝트 초기에 LLM과 관련된 자료구조 문제를 탐색하기 위해 ChatGPT와 Gemini NotebookLM을 활용하였다. 처음에는 RAG, vector search, KV cache, attention, serving 등 다양한 후보를 검토하였다. 그중 가장 관심이 있었던 방향은 “LLM의 연역적 추론 능력을 자료구조 관점에서 어떻게 분석할 수 있는가”였다.

다만 LLM의 추론 능력 자체를 직접 개선하는 것은 모델 학습, fine-tuning, benchmark 평가 등 복잡한 요소가 포함되어 자료구조개론 과제 범위를 벗어난다고 판단하였다. 이에 따라 자연어 규칙을 $A \rightarrow B$ 형태의 방향 edge로 변환하고, 사용자의 query를 시작 노드에서 목표 노드까지의 경로 탐색 문제로 해석하는 방향으로 주제를 구체화하였다.

방향 그래프에서 $A \rightarrow \dots \rightarrow F$ 와 같은 추론 경로를 찾는 기본 아이디어는 직접 제안한 것이며, LLM은 이 아이디어를 KG-RAG, multi-hop reasoning, graph reachability 문제와 연결하여 과제 주제로 다듬는 데 도움을 주었다. 최종적으로 프로젝트 주제는 “KG-RAG 기반 연역 추론에서 Top-K 검색 제한이 방향 그래프의 다중 홉 경로 탐색에 미치는 영향 분석”으로 정하였다.

6.2 자료 조사

주제를 정한 뒤에는 LLM을 활용하여 RAG, KG-RAG, GraphRAG, multi-hop reasoning, Knowledge Graph, graph search 등의 개념을 조사하였다. 특히 일반 RAG가 질문과 유사한 문서 조각을 Top-K로 검색한다는 점과, 다중 홉 추론에서는 질문과 직접적으로 유사하지 않은 중간 정보가 중요할 수 있다는 점을 정리하였다.

또한 Knowledge Graph를 방향 그래프로 해석하고, 연역 추론을 그래프 도달 가능성 문제로 바꾸는 관점을 정리하는 데 LLM을 활용하였다. 이를 통해 본 프로젝트가 단순한 LLM 조사 과제가 아니라, 자연어 지식을 그래프 자료 구조로 변환하고 BFS를 이용해 경로를 탐색하는 구현 과제라는 방향을 명확히 할 수 있었다.

Gemini NotebookLM은 관련 연구 흐름을 확인하는 데 활용하였다. 유사한 연구 분야가 이미 존재한다는 점을 확인하였지만, 본 프로젝트의 목적은 새로운 연구 성과를 내는 것이 아니라 자료구조개론 수준에서 해당 문제를 이해하고 단순화하여 구현하는 것임을 정리하였다.

6.3 데이터셋 생성

실험에는 직접 생성한 synthetic dataset을 사용하였다. LLM을 활용하여 “A 이면 B” 형태의 한국어 규칙 문장을 생성하고, 각 규칙을 source와 target으로 분리할 수 있도록 데이터셋 형식을 설계하였다.

데이터셋에는 정답 경로를 구성하는 answer edge, query와 관련 있어 보이지만 정답 경로에는 포함되지 않는 distractor edge, 그리고 cycle 구조를 만들기 위한 cycle edge를 포함하였다. 이러한 구조는 Top-K 검색이 중간 edge를 누락하여 다중 홉 추론에 실패하는 상황을 실험적으로 재현하기 위한 것이다.

LLM이 생성한 데이터는 그대로 사용하지 않고, source와 target 표현이 expected_path와 정확히 일치하는지 확인하였다. 이는 그래프에서 노드 이름이 조금이라도 다르면 다른 노드로 처리되기 때문이다. 따라서 LLM은 데이터셋 초안을 만드는 데 활용하였고, 최종 데이터 구조와 실험 조건은 직접 검토하며 수정하였다.

6.4 구현 및 디버깅

구현 과정에서는 CSV 파일 읽기, adjacency list 기반 KG 생성, BFS 경로 탐색, Top-K edge 선택, 1-hop Expansion 구현, 결과 CSV 저장 및 시각화 코드 작성 과정에서 LLM을 활용하였다. 특히 Naive Top-K 방식이 작은 K에서 실패하는 원인을 분석할 때, 전체 KG에서는 성공하지만 부분 KG에서는 실패한다는 점을 확인하고, 정답 edge 보존율과 누락 edge 수를 추가 지표로 기록하도록 코드를 수정하였다. 이를 통해 실패 원인이 BFS 자체가 아니라 검색 단계에서 필요한 중간 edge가 선택되지 않았기 때문임을 확인할 수 있었다.

6.5 결과 해석

실험 결과를 해석하는 과정에서도 LLM을 활용하였다. 특히 Naive Top-K 방식이 K=88 부근에서야 성공률 1.00에 도달하고, Top-K + 1-hop Expansion 방식은 K=6 수준에서 성공률 1.00에 도달한 결과를 어떻게 설명할지 정리하는 데 도움을 받았다.

해석 과정에서 가장 중요했던 점은 “전체 KG에 정답이 존재한다”는 사실과 “Top-K 부분 KG에 정답 경로가 보존된다”는 사실이 다르다는 점이었다. 전체 KG에서는 모든 query에 대해 정답 경로가 존재했지만, Naive Top-K 방식에서는 중간 edge가 누락되어 경로가 단절되는 경우가 많았다. LLM과의 대화를 통해 이 현상을 RAG의 Top-K 검색 제한이 다중 홉 연역 추론에 취약할 수 있는 사례로 정리할 수 있었다.

또한 단순히 성공률만 비교하는 것이 아니라, 정답 edge 보존율과 평균 누락 edge 수를 함께 분석해야 한다는 점도 확인하였다. 이를 통해 Naive Top-K 방식의 실패 원인이 단순한 탐색 알고리즘의 문제가 아니라, 검색 단계에서 필요한 edge가 선택되지 않았기 때문임을 설명할 수 있었다.

Top-K + 1-hop Expansion 방식의 결과 해석에서도 LLM을 활용하였다. Expansion 방식은 Naive Top-K보다 더 많은 edge를 선택하므로 탐색 공간이 증가할 수 있지만, 훨씬 작은 K에서도 정답 경로를 복원할 수 있었다. 이를 통해 그래프 연결성을 활용한 확장이 다중 홉 추론에서 중간 edge 누락 문제를 완화할 수 있다는 결론을 정리하였다.

결과적으로 LLM은 실험 결과를 단순 수치로 보는 데 그치지 않고, 이를 자료구조적 의미로 해석하는 데 도움을 주었다. 다만 프로젝트의 핵심 문제의식인 연역적 추론에 대한 관심, 자연어 규칙을 방향 그래프로 보고 경로를 탐색한다는 기본 아이디어, 그리고 세 방식의 비교 실험 구조는 직접 고민하며 구성하였다.